

Desafio Técnico - API REST para Livraria

Construir uma API REST moderna utilizando **.NET 8**, com foco em arquitetura, boas práticas de desenvolvimento backend, observabilidade, segurança e performance.

Visão Geral do Projeto

A aplicação deve permitir o gerenciamento completo de livros de uma livraria, incluindo criação, consulta, atualização e exclusão lógica de registros. O projeto deve seguir boas práticas de arquitetura, segurança, validação, cache distribuído e observabilidade.

Requisitos Funcionais

- Criar livro
- Listar livros
- Buscar livro por ID
- Atualizar livro
- Excluir livro
- Filtros por título, autor, gênero e faixa de preço
- Paginação
- Ordenação por título, preço e data de criação

Tecnologias e Ferramentas

- .NET 8
- Entity Framework Core
- SQL Server
- FluentValidation
- Serilog
- Redis Cache

- Docker
- JWT Authentication
- xUnit + Moq
- Postman
- Health Checks
- Repository Pattern
- Unit of Work

Endpoints Obrigatórios

Método	Endpoint	Descrição
POST	/api/books	Criar livro
GET	/api/books	Listar livros
GET	/api/books/{id}	Buscar livro por ID
PUT	/api/books/{id}	Atualizar livro
DELETE	/api/books/{id}	Excluir livro
GET	/api/books/search	Buscar por título, autor ou gênero
PATCH	/api/books/{id}/stock	Atualizar estoque
PATCH	/api/books/{id}/price	Atualizar preço

Campos Obrigatórios

Campo	Tipo	Regra
Id	Guid	Gerado automaticamente
Title	string	2 a 120 caracteres
Author	string	2 a 120 caracteres
Genre	string/enum	Deve ser válido
Price	decimal	Maior ou igual a 0
Stock	int	Maior ou igual a 0
CreatedAt	DateTime	Preenchido na criação
UpdatedAt	DateTime?	Atualizado nas alterações
ISBN	string	Obrigatório e único
Publisher	string	Obrigatório

PublishedYear	int	Não pode ser maior que o ano atual
IsActive	bool	Usado para soft delete
Description	string	Opcional, máximo 1000 caracteres

Regras de Negócio

- Não permitir livro duplicado com mesmo Title + Author
- Não permitir ISBN duplicado
- Price não pode ser negativo
- Stock não pode ser negativo
- Genre deve ser um enum válido
- PublishedYear não pode ser futuro
- Exclusão deve ser soft delete
- Listagens devem retornar apenas livros ativos
- Não permitir atualizar livro inativo
- Preencher UpdatedAt nas alterações
- Invalidar cache ao criar, atualizar ou excluir livros
- Apenas Admin pode criar, atualizar e excluir livros
- Implementar Rate Limiting
- Implementar Correlation ID
- Implementar auditoria de alterações

Health Checks

Criar o endpoint **GET /health** para monitoramento da aplicação, validando:

- API online
- Conexão com SQL Server
- Conexão com Redis

Redis Cache

Utilizar Redis Cache para otimização de consultas e melhoria de performance.

- GET /api/books
 - GET /api/books/{id}
 - GET /api/books/search
 - Cache da listagem por 5 minutos
 - Cache do livro por ID por 10 minutos
 - Invalidar cache ao criar, atualizar ou excluir livros
 - Utilizar chaves padronizadas
-

Objetivo:

Demonstrar conhecimentos em desenvolvimento backend moderno com .NET, arquitetura em camadas, segurança, observabilidade, cache distribuído, testes automatizados e boas práticas de engenharia de software.